

Documentación Técnica Detallada del Servicio Notificaciones (WebSocket + Redis)

Descripción General

El servicio **Notificaciones** es un microservicio desarrollado en **Node.js** que forma parte del sistema **Campus Universitario Inteligente (C5 Campus)**. Su función principal es distribuir alertas de emergencia en **tiempo real** hacia los operadores del sistema C5, permitiendo que la información generada por otros microservicios llegue de forma inmediata al centro de monitoreo.

Este servicio recibe mensajes desde **Redis**, mantiene conectados a los operadores mediante **WebSockets** y envía las alertas activas a todos los usuarios que estén visualizando el panel de monitoreo. Además, cuando no existen operadores conectados, almacena las alertas en una **cola de pendientes** para entregarlas automáticamente cuando un operador vuelva a conectarse.

Su función principal es:

- Recibir alertas de emergencia desde una cola Redis.
- Distribuir alertas en tiempo real a los operadores C5 conectados.
- Mantener comunicación bidireccional mediante WebSockets.
- Guardar alertas pendientes cuando no hay operadores activos.
- Entregar automáticamente las alertas pendientes al reconectarse un operador.
- Recibir transcripciones desde otros módulos del sistema y enviarlas al panel de monitoreo.
- Exponer un endpoint de salud para comprobar que el microservicio funciona correctamente.

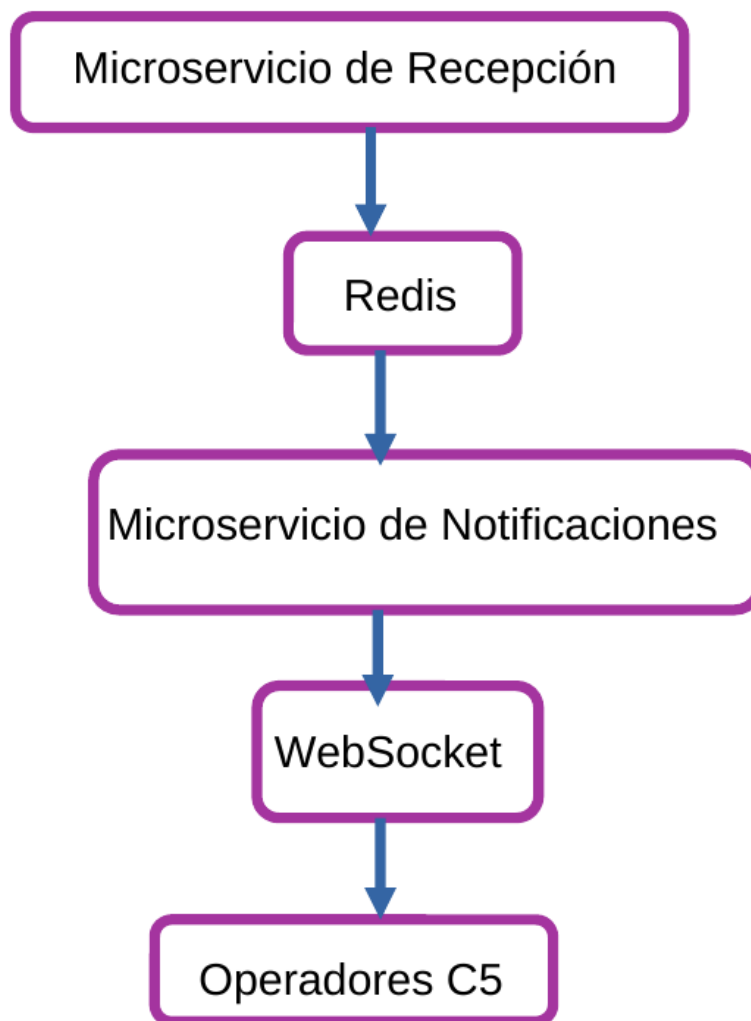
Tecnologías utilizadas

Para implementar el microservicio se utilizaron tecnologías enfocadas en comunicación en tiempo real, procesamiento asíncrono y despliegue mediante contenedores.

Tecnología	Función dentro del servicio
Node.js	Permite desarrollar la lógica principal del microservicio.
Express	Permite crear la API REST y exponer el endpoint de estado /health.
WebSocket (ws)	Mantiene comunicación en tiempo real entre el servidor y los operadores C5 conectados.
Redis / ioredis	Gestiona colas de mensajes, alertas pendientes y suscripción a canales de transcripción.
Docker	Permite empaquetar y ejecutar el servicio dentro de un contenedor aislado y portable.

Arquitectura general del servicio

La arquitectura del microservicio de **Notificaciones** se basa en un flujo asíncrono. Primero, otro microservicio genera o procesa una alerta y la coloca en **Redis**. Después, el servicio **Notificaciones** escucha la cola correspondiente y distribuye el mensaje mediante **WebSocket** a los operadores conectados.

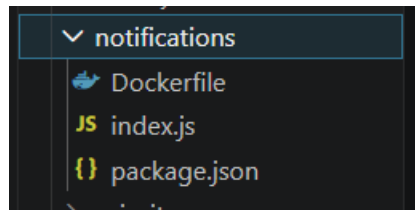


Arquitectura general: las alertas pasan por Redis, el servicio Notificaciones las procesa y los operadores C5 las reciben mediante WebSocket.

Gracias a esta arquitectura, el sistema no depende de que los operadores estén conectados en el momento exacto en que se genera la alerta. Si no existen clientes activos, el mensaje se conserva en una cola de pendientes y se entrega posteriormente. Esto evita la pérdida de información y mejora la confiabilidad del sistema.

Estructura de la carpeta de Notificaciones

El microservicio de Notificaciones se organiza en una estructura sencilla, similar a la utilizada por otros servicios del sistema C5 Campus. Cada archivo cumple una función específica dentro del despliegue y ejecución del servicio.



Dockerfile

El archivo **Dockerfile** define el entorno donde se ejecutará el microservicio de **Notificaciones**. Su propósito es automatizar la creación de una imagen **Docker** que incluya Node.js, las dependencias necesarias y el código fuente del proyecto.

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package.json .
4 RUN npm install --production
5 COPY . .
6 CMD ["node", "index.js"]
7
```

La instrucción **FROM node:18-alpine** establece la imagen base sobre la cual se construirá el contenedor. En este caso se utiliza **Node.js 18** junto con **Alpine Linux**, una distribución ligera que ayuda a reducir el tamaño final de la imagen y permite que el servicio se ejecute con menor consumo de recursos.

Posteriormente, **WORKDIR /app** define el directorio principal de trabajo dentro del contenedor. Todas las instrucciones siguientes se ejecutan tomando como referencia esta carpeta, lo que facilita mantener ordenados los archivos de la aplicación.

La instrucción **COPY package.json .** copia primero el archivo de dependencias. Este paso permite aprovechar la caché de Docker, ya que si solo cambia el código fuente, Docker puede reutilizar la capa donde ya se instalaron las dependencias.

Después, **RUN npm install --production** instala únicamente los paquetes necesarios para ejecutar el servicio. Finalmente, **COPY . .** copia el resto del proyecto y **CMD ["node", "index.js"]** indica el comando que se ejecutará automáticamente cuando el contenedor se inicie.

Package.json

El archivo **package.json** es el archivo de configuración principal del microservicio. Define la información básica del proyecto, el archivo de arranque, los comandos de ejecución y las dependencias externas que necesita la aplicación.

```

services > notifications > {} package.json > ...
...
1  {
2    "name": "c5-notifications",
3    "version": "1.0.0",
4    "main": "index.js",
   ▶ Depurar
5    "scripts": { "start": "node index.js" },
6    "dependencies": {
7      "express": "^4.18.2",
8      "ws": "^8.16.0",
9      "ioredis": "^5.3.2"
10   }
11 }
12

```

La propiedad **name** permite identificar el servicio dentro del sistema como **c5-notifications**. La propiedad **main** indica que el archivo principal de ejecución es **index.js**, por lo que toda la lógica de conexión con Redis, WebSocket y la API REST se inicia desde ese archivo.

La sección **scripts** incluye el comando **start**, el cual ejecuta **node index.js**. Esto facilita iniciar el microservicio de forma local o dentro de Docker usando un comando estándar.

En la sección **dependencies** se incluyen las bibliotecas necesarias para el funcionamiento del servicio. **Express** permite crear la API REST, **ioredis** permite conectarse con Redis y **ws** proporciona la funcionalidad **WebSocket** para mantener conectados a los operadores en tiempo real.

Index.js

El archivo **index.js** es el componente principal del microservicio de **Notificaciones**, ya que contiene la configuración del servidor, la conexión con **Redis**, la administración de operadores conectados, la distribución de alertas y la exposición del endpoint de salud.

Al iniciar la aplicación se importan las dependencias principales: Express para la API REST, http para crear el servidor base, ws para habilitar WebSockets e ioredis para comunicarse con Redis. De esta forma, el mismo servicio puede recibir datos desde colas, mantener clientes conectados y responder solicitudes REST.

```

services > notifications > JS index.js > ...
...
1 // services/notifications/index.js
2 const express = require("express");
3 const http = require("http");
4 const WebSocket = require("ws");
5 const Redis = require("ioredis");
6
7 const app = express();
8 app.use(express.json());
9

```

Configuración inicial

El servicio define un puerto de ejecución mediante la variable **PORT**. Si no existe una variable de entorno configurada, el sistema utiliza el puerto **3004** como valor predeterminado.

```

10 const PORT      = process.env.PORT      || 3004;
11 const REDIS_URL = process.env.REDIS_URL || "redis://localhost:6379";
12
13 const redis      = new Redis(REDIS_URL);
14 const redisPub   = new Redis(REDIS_URL);
15 const redisSub   = new Redis(REDIS_URL); // canal de transcripción
16
17 const server = http.createServer(app);
18 const wss    = new WebSocket.Server({ server });
19
20 // — WebSocket connections —————
21 const operators = new Set();
22
23 wss.on("connection", (ws, req) => {
24   operators.add(ws);
25   console.log(`[Notifications] Operador conectado. Total: ${operators.size}`);
26
27   ws.send(JSON.stringify({
28     type: "connected",
29     message: "Conectado al sistema C5 de alertas",
30     timestamp: new Date().toISOString(),
31   }));
32
33   ws.on("close", () => {
34     operators.delete(ws);
35     console.log(`[Notifications] Operador desconectado. Total: ${operators.size}`);
36   });
37

```

También se configura la dirección de **Redis** mediante **REDIS_URL**. Esta variable permite que el microservicio se conecte al servidor **Redis** local o a un contenedor **Redis** dentro de una arquitectura distribuida.

```

38   ws.on("error", (err) => {
39       console.error("[Notifications] WebSocket error:", err.message);
40       operators.delete(ws);
41   });
42 });
43
44 // Reenviar alertas pendientes cuando se conecta un operador
45 wss.on("connection", async (ws) => {
46     const pending = await redis.llen("queue:notify_pending");
47     if (pending > 0) {
48         console.log(`[Notifications] Enviando ${pending} alertas pendientes al nuevo operador`);
49         const items = await redis.lrange("queue:notify_pending", 0, -1);
50         for (const item of items) {
51             if (ws.readyState === WebSocket.OPEN) {
52                 ws.send(JSON.stringify({ type: "pending_alert", data: JSON.parse(item) }));
53             }
54         }
55         await redis.del("queue:notify_pending");
56     }
57 });
58
59 // — Broadcast a todos los operadores —————
60 function broadcastToOperators(type, data) {
61     const message = JSON.stringify({ type, data });
62     let sent = 0;
63     for (const ws of operators) {
64         if (ws.readyState === WebSocket.OPEN) {
65             ws.send(message);
66             sent++;
67         }
68     }
69     console.log(`[Notifications] [${type}] enviado a ${sent} operador(es)`);
70     return sent;
71 }
72

```

Cientes Redis

La conexión con Redis se divide en tres clientes diferentes para separar responsabilidades. Esta separación mejora la claridad del código y evita conflictos entre operaciones de cola, publicación y suscripción.

```
// — Procesador de cola Redis (alertas) —
async function processQueue() {
  console.log("[Notifications] Escuchando cola queue:notify...");
  while (true) {
    try {
      const result = await redis.brpop("queue:notify", 5);
      if (!result) continue;

      const alert = JSON.parse(result[1]);
      const delivered = broadcastToOperators("alert", alert);

      if (delivered === 0) {
        await redisPub.lpush("queue:notify_pending", JSON.stringify(alert));
        console.log(`[Notifications] Sin operadores – alerta guardada en pendientes`);
      }
    } catch (err) {
      console.error("[Notifications] Error procesando:", err.message);
      await new Promise((r) => setTimeout(r, 1000));
    }
  }
}

// — Suscripción a transcripciones —
redisSub.subscribe("channel:transcription", (err) => {
  if (err) {
    console.error("[Notifications] Error suscribiéndose a transcripción:", err.message);
  } else {
    console.log("[Notifications] Suscrito a channel:transcription");
  }
});

redisSub.on("message", (channel, message) => {
  if (channel === "channel:transcription") {
    try {
      const data = JSON.parse(message);
      console.log(`[Notifications] Transcripción recibida para alerta ${data.alert_id}`);
    }
  }
});
```

Cliente Redis	Función
redis	Escucha y procesa la cola queue:notify donde llegan las alertas.
redisPub	Permite publicar mensajes o dejar preparada la comunicación con otros servicios.
redisSub	Se suscribe al canal channel:transcription para recibir transcripciones.

Servidor WebSocket y operadores conectados

El servicio crea un servidor **WebSocket** sobre el servidor **HTTP**. Esto permite mantener una conexión persistente con los **operadores C5** y enviarles mensajes de forma inmediata sin que tengan que consultar constantemente al servidor.

Los operadores conectados se almacenan en un conjunto llamado **operators**. Cuando un operador se conecta, el sistema lo agrega a la lista, registra la conexión y envía un mensaje de bienvenida confirmando que se conectó correctamente al sistema de alertas.

```
112     }
113   }
114 });
115
116 // — REST API —
117 app.get("/health", (req, res) =>
118   res.json({ status: "ok", service: "notifications", operators: operators.size })
119 );
120
121 server.listen(PORT, () => {
122   console.log(`[Notifications] Servicio iniciado en puerto ${PORT}`);
123   console.log(`[Notifications] WebSocket disponible en ws://localhost:${PORT}`);
124   processQueue();
125 });
```

Alertas pendientes

Una función importante del microservicio es evitar que las alertas se pierdan cuando no hay operadores conectados. Para ello utiliza la cola **queue:notify_pending**, donde se almacenan temporalmente las alertas que no pudieron entregarse en el momento en que fueron recibidas.

Cuando un operador se conecta nuevamente, el sistema revisa si existen alertas pendientes. Si las encuentra, las recupera, las envía al operador y posteriormente las elimina de la cola. Esto permite conservar la información y garantiza que las emergencias generadas durante una desconexión puedan ser revisadas.

1. El operador abre el panel de monitoreo y establece una conexión WebSocket.
2. El sistema agrega el operador al conjunto de clientes activos.
3. Se consulta la cola **queue:notify_pending**.
4. Las alertas almacenadas se envían al operador recién conectado.
5. La cola se limpia para evitar duplicados.

Distribución de mensajes con **broadcastToOperators()**

La función **broadcastToOperators()** se encarga de enviar un mismo mensaje a todos los operadores conectados. Esta función construye el mensaje en formato **JSON**, recorre el

conjunto de operadores activos, verifica que cada conexión esté abierta y transmite la información.

```
broadcastToOperators(type, data)
```

Proceso:

1. Construye el mensaje.
2. Recorre todos los operadores conectados.
3. Envía la información por WebSocket.
4. Cuenta cuántos operadores recibieron el mensaje.

Un ejemplo de mensaje enviado a los operadores es una alerta con su zona y prioridad. Este formato permite que el panel de monitoreo identifique el tipo de mensaje y muestre los datos relevantes de manera clara.

```
{
  "type": "alert",
  "data": {
    "zona": "Biblioteca",
    "prioridad": "CRITICA"
  }
}
```

Procesamiento de alertas con processQueue()

La función **processQueue()** es el proceso principal del microservicio. Su tarea es escuchar permanentemente la cola Redis **queue:notify**, donde otros servicios colocan las alertas que deben enviarse al centro de monitoreo.

```
const result = await redis.brpop("queue:notify", 0);
```

La instrucción **brpop** permite esperar una alerta de manera bloqueante. Esto significa que el servicio permanece atento a la cola sin consumir recursos de forma innecesaria y responde inmediatamente cuando aparece un nuevo mensaje.

Cuando se recibe una alerta, el sistema la convierte desde **JSON**, la envía a todos los operadores conectados y verifica si alguien la recibió. Si no existe ningún operador activo, la alerta se guarda en **queue:notify_pending** para entregarla posteriormente.

1. Espera una alerta en la cola queue:notify.
2. Obtiene y convierte el mensaje recibido.
3. Envía la alerta a los operadores conectados mediante WebSocket.
4. Si no hay operadores activos, almacena la alerta en queue:notify_pending.
5. Continúa escuchando nuevas alertas de manera permanente.

Recepción de transcripciones

El microservicio también recibe mensajes provenientes del módulo de reconocimiento de voz. Para lograrlo se suscribe al canal **channel:transcription** mediante **Redis Pub/Sub**. Cuando llega una transcripción, el sistema la procesa y la envía a los operadores conectados.

```
await redisSub.subscribe("channel:transcription");

redisSub.on("message", (channel, message) => {
  const transcription = JSON.parse(message);
  broadcastToOperators("transcription", transcription);
});
```

Este mecanismo permite que los operadores no solo reciban alertas estructuradas, sino también información textual generada por otros módulos del sistema, como mensajes de voz convertidos a texto.

```
{
  "alert_id": "A001",
  "texto": "Me están asaltando"
}
```

API REST - Endpoint Health

Además de WebSocket y Redis, el microservicio expone una ruta **REST** para comprobar su estado. Esta ruta permite verificar que el servicio se encuentre funcionando correctamente y consultar cuántos operadores están conectados en ese momento.

Ruta	Método	Función
/health	GET	Devuelve el estado del servicio, el nombre del microservicio y el número de operadores conectados.

```
{
  "status": "ok",
  "service": "notifications",
  "operators": 3
}
```

Esta respuesta es útil para pruebas, monitoreo y verificación del despliegue, ya que confirma que el servidor está activo y que el módulo WebSocket está administrando conexiones de operadores.

Mensajes principales del servicio

El microservicio trabaja con diferentes tipos de mensajes. Cada mensaje incluye un campo **type** que indica al cliente cómo debe interpretar la información recibida.

Tipo de mensaje	Origen	Propósito
connected	Servidor WebSocket	Confirma que el operador se conectó correctamente al sistema.
alert	Cola queue:notify	Envía una alerta de emergencia a los operadores conectados.
transcription	Canal channel:transcription	Distribuye texto proveniente del reconocimiento de voz.
health	Endpoint /health	Permite verificar el estado general del microservicio.

Flujo de operación

El flujo de operación resume el comportamiento del microservicio desde que se genera una alerta hasta que los operadores la visualizan en el sistema C5.

1. Se genera una alerta desde otro microservicio del sistema.
2. Redis almacena la alerta en la cola queue:notify.
3. El microservicio de Notificaciones escucha la cola y recibe el mensaje.
4. La alerta se convierte a JSON para poder procesarla.
5. El servicio revisa cuántos operadores están conectados mediante WebSocket.
6. Si existen operadores activos, la alerta se envía inmediatamente.
7. Si no hay operadores conectados, la alerta se almacena en queue:notify_pending.
8. Cuando un operador se conecta, recibe las alertas pendientes.
9. El sistema continúa monitoreando nuevas alertas y transcripciones.

Resultados obtenidos

Durante las pruebas, el microservicio demostró que puede integrarse correctamente dentro de una arquitectura distribuida y distribuir información en tiempo real hacia el centro de monitoreo.

- Se logró recibir alertas desde Redis mediante la cola queue:notify.
- Se enviaron alertas en tiempo real mediante WebSockets.
- Se mantuvieron múltiples operadores conectados simultáneamente.
- Se almacenaron alertas pendientes cuando no existían operadores activos.
- Se recuperaron y entregaron alertas pendientes al conectarse un operador.
- Se recibieron transcripciones desde otros microservicios mediante Redis Pub/Sub.
- Se expuso el endpoint /health para verificar el estado del servicio.
- Se integró correctamente dentro de la arquitectura distribuida del sistema C5 Campus.